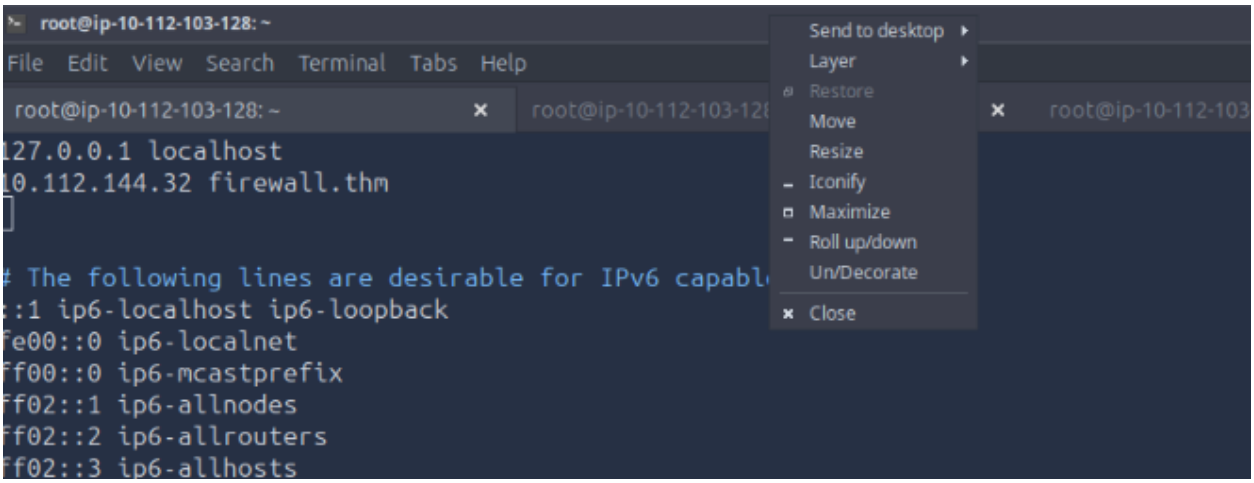
Jr.Pen 11.5. Checkmate

Challenge: Marco Bianchi, a systems administrator, recently deployed several internal services, including a firewall console, employee portal, social platform, and SSH access to critical infrastructure. Due to tight deadlines and operational pressure, Marco reused weak, predictable, and pattern-based passwords across multiple systems. Your objective is to conduct a password security assessment to identify weaknesses in Marco's authentication practices.

Before starting the room, I clicked the link provided to me and added firewall.thm to the /etc/hosts file to use the system's static hostname lookup table.

- `/etc/hosts`: A system file that maps IP addresses to hostnames (domain names) locally.



Answer the questions below

> What is the password for Level 1?

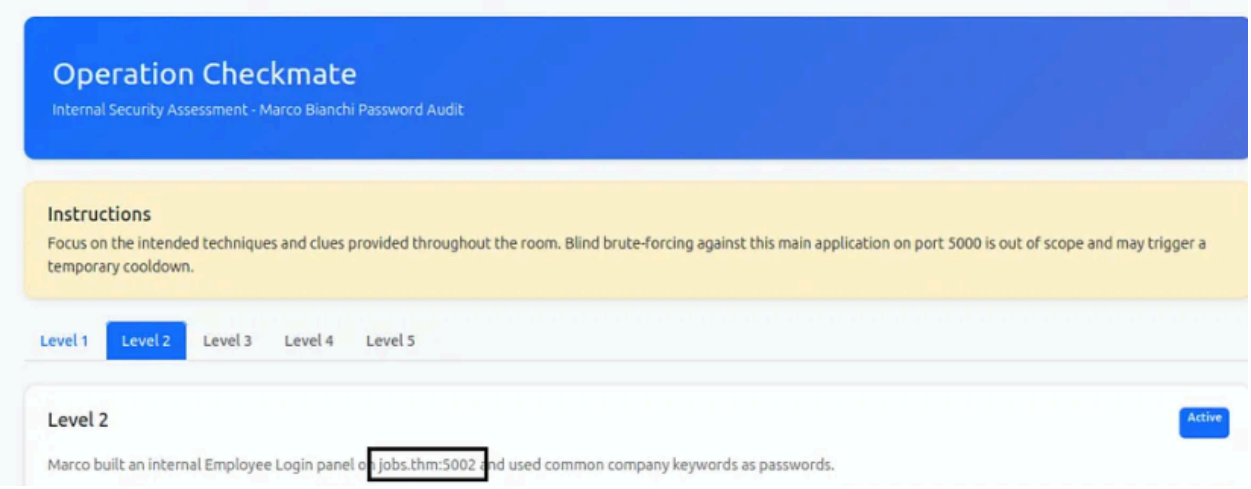
12345



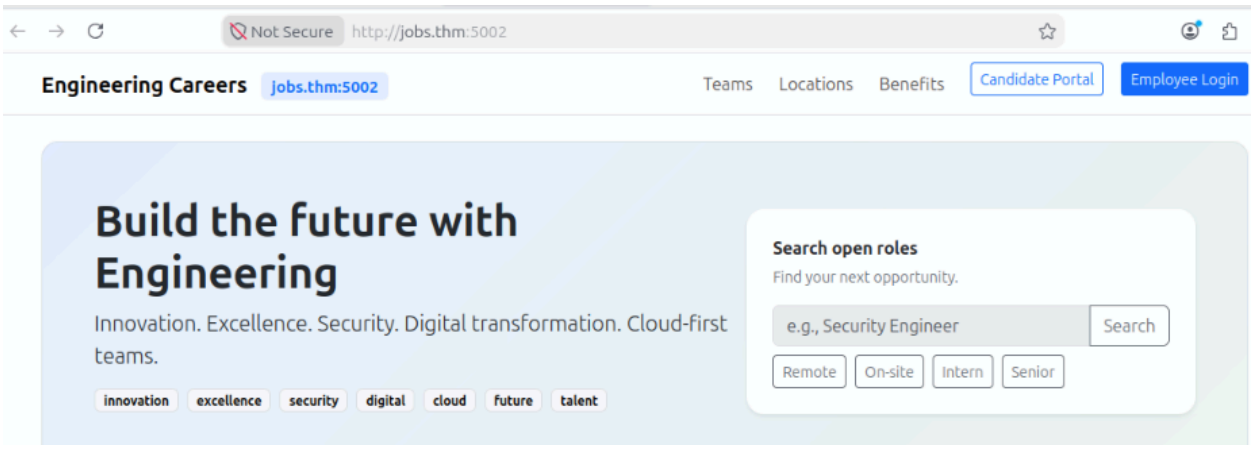
As room says, I tried all the possibilities at random. (like admin - admin , admin - password). Once we log in with the default password 12345, the site forwards us to this page.

> What is the password for Level 2?

excellence

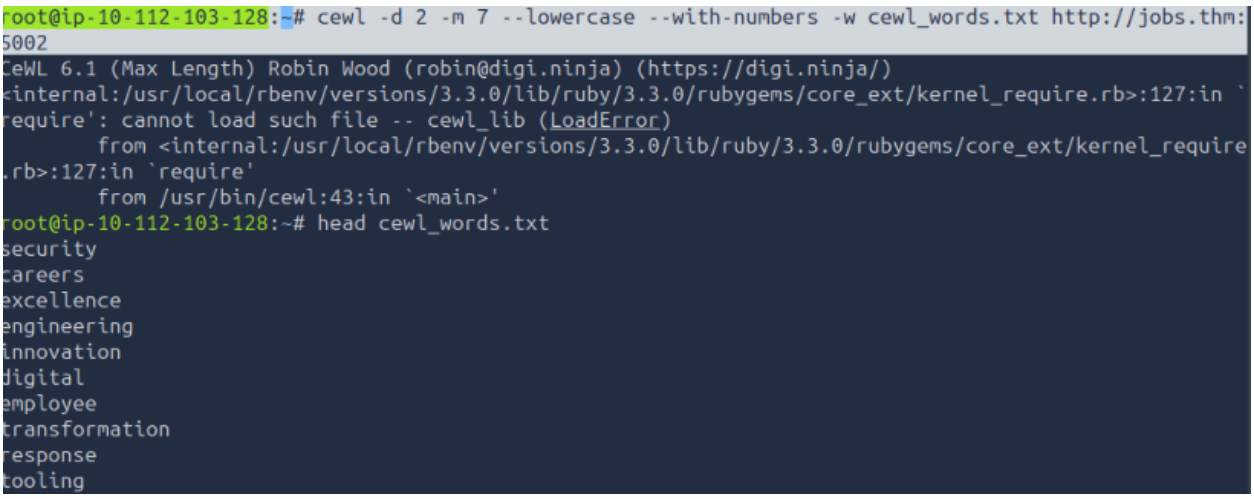


After adding jobs.thm to /etc/hosts, I accessed the page and attempted the password listed on it.



We need to build a custom wordlist from the website's content and use it to brute-force the login page, using tools such as CeWL, curl, or wget.

run cewl command

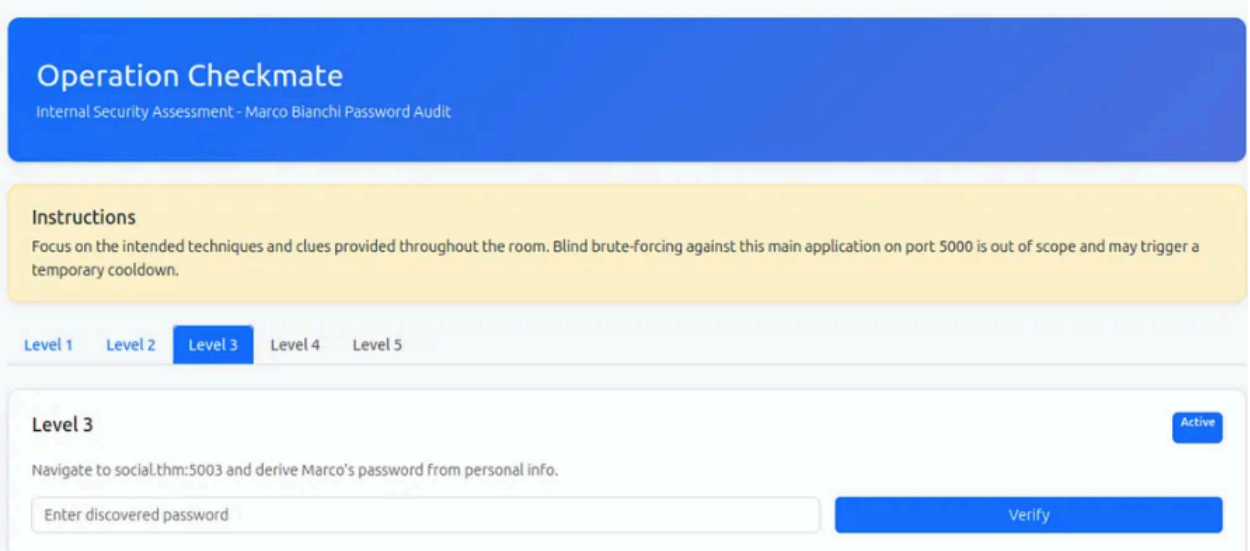



```
root@ip-10-112-103-128:~# cewl -d 2 -m 7 --lowercase --with-numbers -w cewl_words.txt http://jobs.thm:5002
```

cewl: A tool that crawls websites to build custom wordlists for password testing. `-d 2`: Sets the **depth** of the web crawl to `2`. CeWL will start at the target URL, `-m 7`: Sets the **minimum word length** to `7` characters.

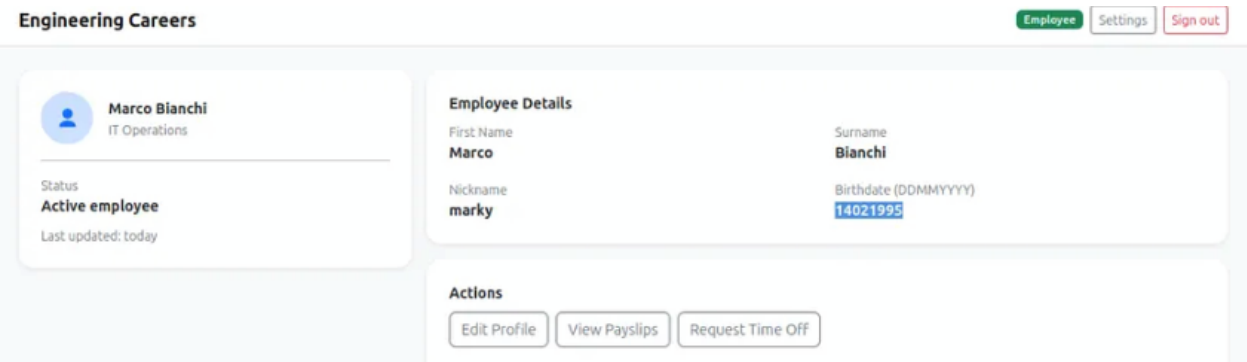
> What is the password for Level 3?

Bianchi2495



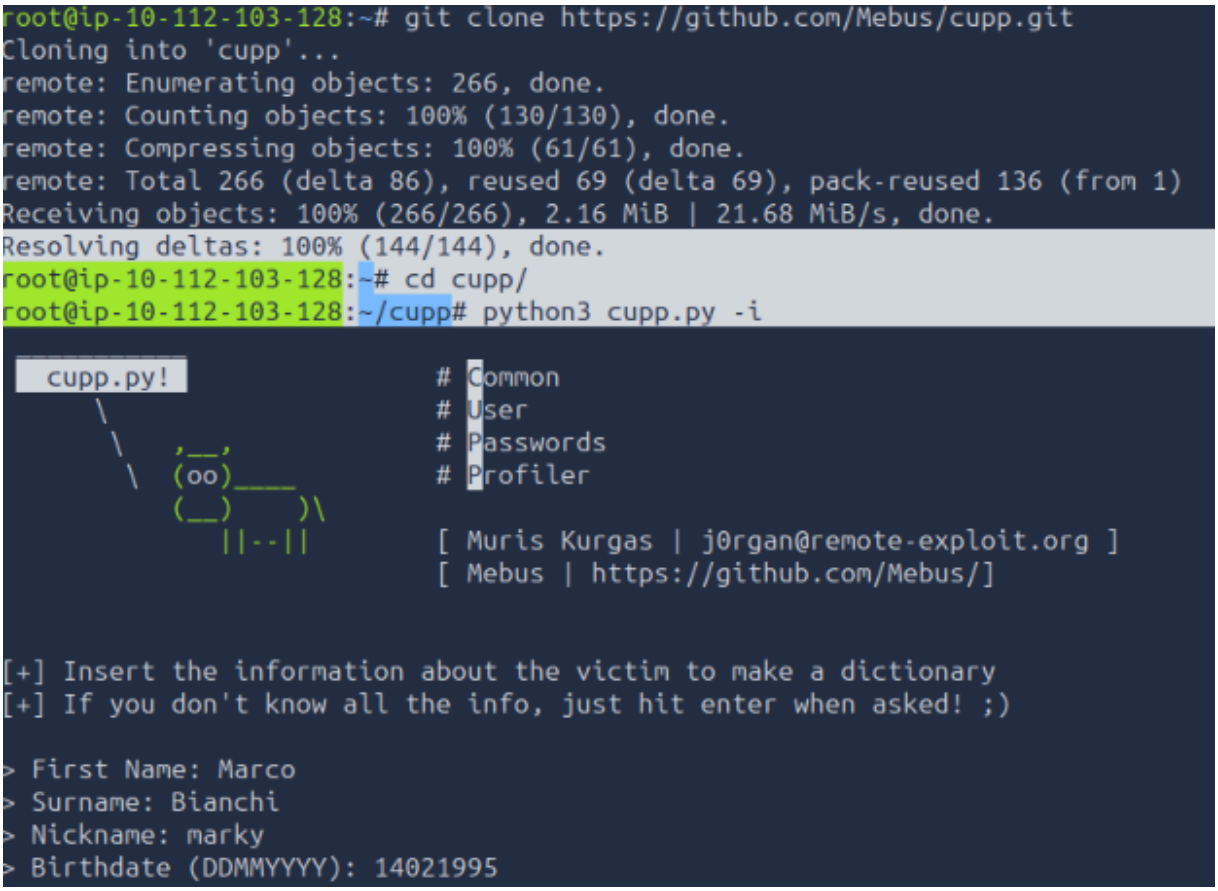
After adding social.thm to /etc/hosts, I accessed the page and cloned the Mebus GitHub repository.

We'll use CUPP to build a targeted wordlist based on Marco's personal details.




```
root@ip-10-112-103-128:~# git clone https://github.com/Mebus/cupp.git
```

This command clones the official GitHub repository for **CUPP** (Common User Passwords Profiler), a tool used to generate custom password lists based on target profiling data.

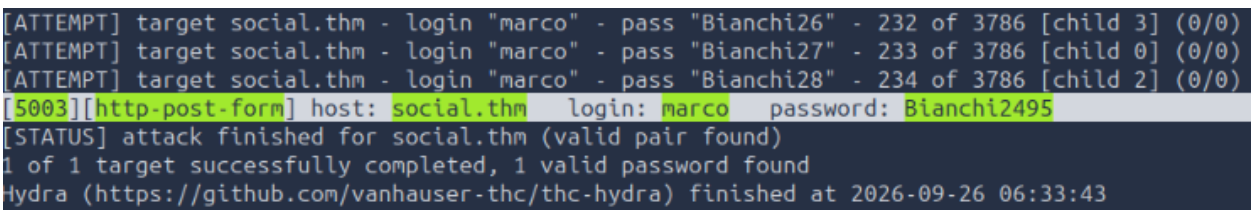


After generating the personalized wordlist, we will use it to brute-force the login page of the social platform using **Hydra**.

run hydra

```
root@ip-10-112-103-128:~/cupp# hydra -l marco -P marco.txt -f
-V -t 4 social.thm http-post-form '/login:username=^USER^&pas
sword=^PASS^:F=Invalid' -s 5003
```

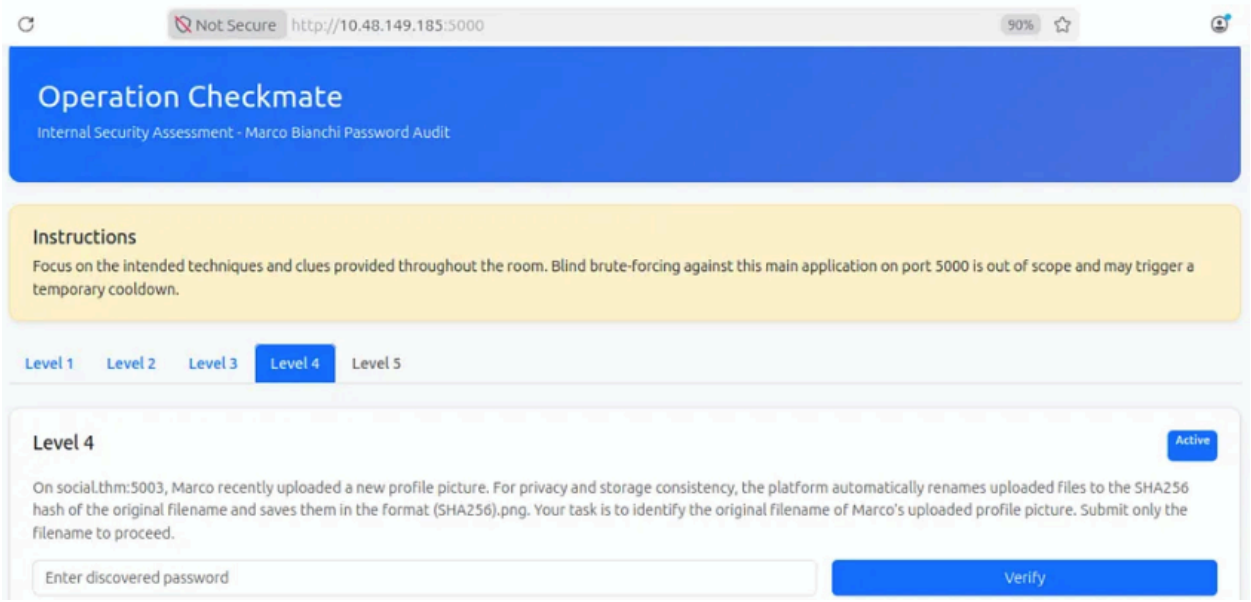
This command uses **Hydra**, a multi-threaded network logon cracker, to attempt a dictionary-based attack against a web-based login form running on host `social.thm` at port `5003`.



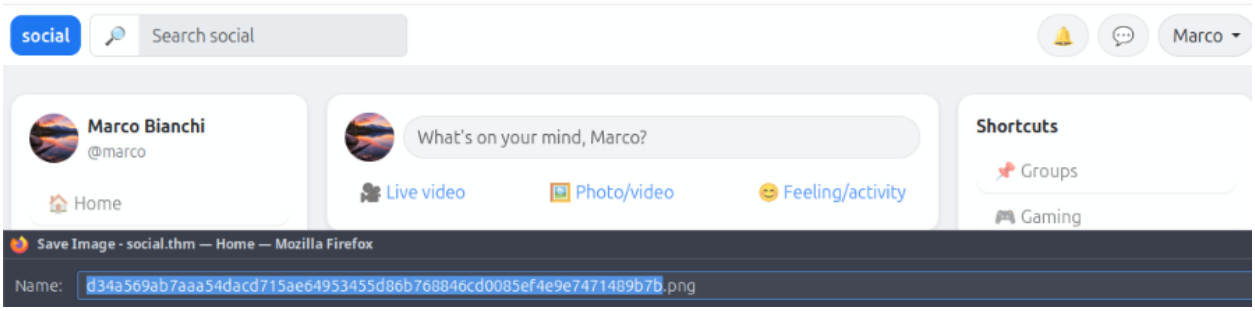
- `hydra`: Invokes the Hydra application.
- `l marco`: Specifies a single static username (`marco`) to target.
- `P marco.txt`: Specifies the wordlist file containing candidate passwords to test against the targeted account.
- `f`: Tells Hydra to exit immediately after finding the first valid password pair.
- `V`: Enables verbose output, showing each username and password attempt in real time in the terminal.
- `t 4`: Limits Hydra to `4` parallel tasks (threads) to avoid overloading the target web server or triggering strict rate limiting.
- `social.thm`: The domain name or IP address of the target host.
- `http-post-form`: Specifies the module used to handle HTTP POST request forms.
- `'/login:username=^USER^&password=^PASS^:F=Invalid'`: Defines the target URL path, HTTP body parameters, and success/failure conditions:
  - `/login`: The URL path where the login form processes requests.
  - `username=^USER^&password=^PASS^`: The POST body syntax, where `^USER^` is replaced by `marco` and `^PASS^` is replaced by entries from `marco.txt`.
  - `F=Invalid`: The failure condition. Hydra checks the HTTP response for the string `"Invalid"`. If that string is present, Hydra assumes the login failed; if it is missing, Hydra marks the attempt as successful.
- `s 5003`: Sets the target port to `5003` instead of the default HTTP port (`80`).

> What is the password for Level 4?

family

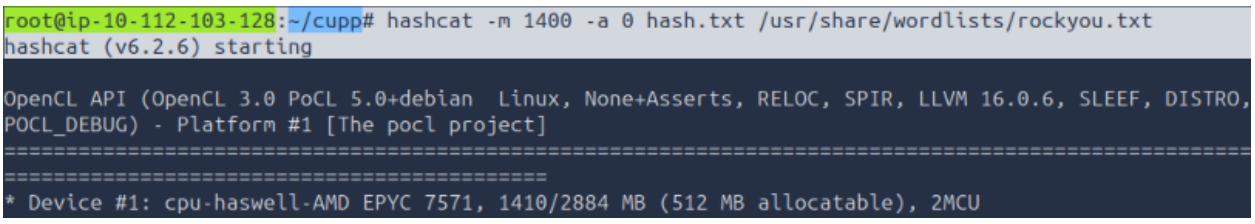


After logged in the page , clicked the picture then save as



create vim hash.txt file

```
root@ip-10-112-103-128:~/cupp# hashcat -m 1400 -a 0 hash.txt /usr/share/wordlists/rockyou.txt
```

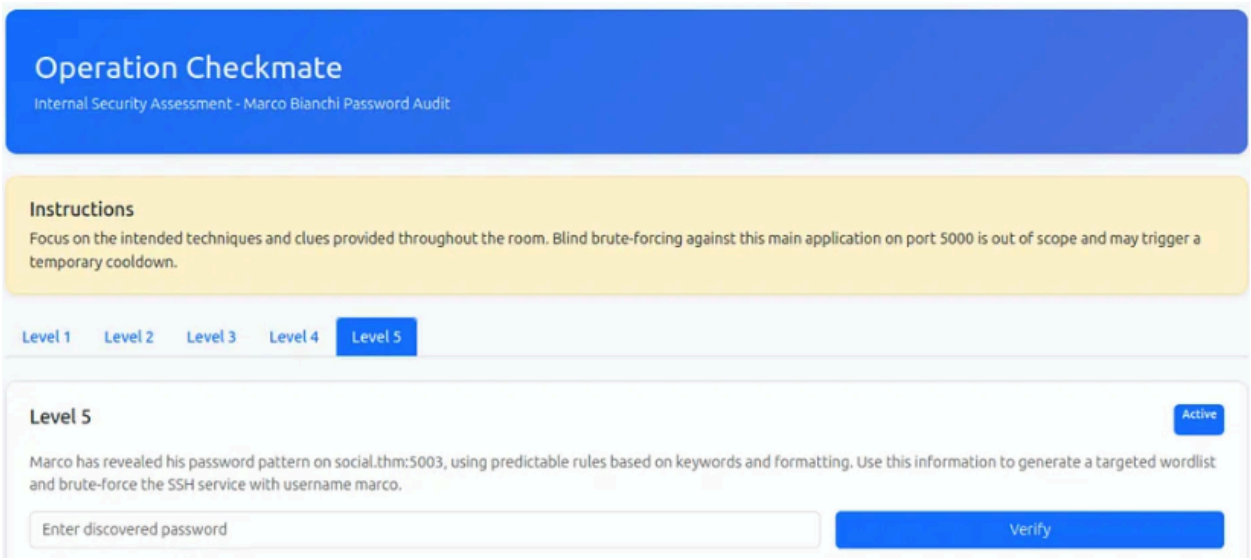


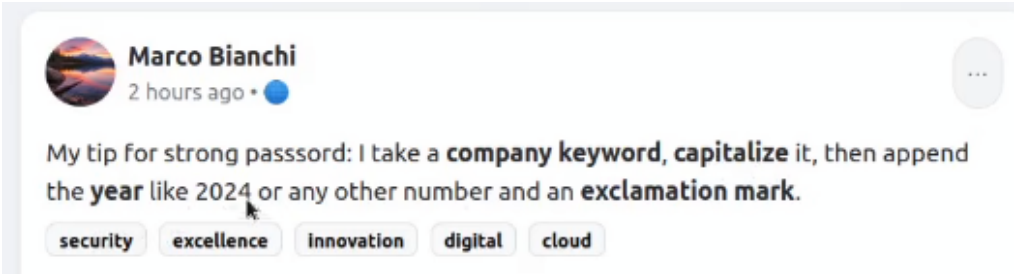
- `m 1400`: Specifies the **hash type** algorithm. Mode `1400` tells Hashcat to process the target hashes as **SHA256**.

- `a 0`: Sets the **attack mode**. Mode `0` corresponds to a **Straight / Dictionary attack**, where Hashcat tests words directly from a provided wordlist line by line.
- `hash.txt`: The file containing the raw hash(es) to be cracked.
- `/usr/share/wordlists/rockyou.txt`: The path to the wordlist file containing candidate passwords.

> What is the password for Level 5?

Security2024!





We'll use Crunch to generate a wordlist that matches the specified password format.

```
crunch 13 13 -t Security20%% ! -o security_wordlist.txt
```

- `crunch`: Invokes the Crunch utility.
- `13 13`: Sets both the **minimum length** and **maximum length** of the generated words to **13 characters**.
- `t Security20%% !`: Specifies a fixed pattern template (`t`) for generating words. In Crunch syntax, specific characters serve as placeholders for dynamic variations:
  - **Literal characters:** `Security20` remains static at the beginning of every word.
  - `%`: Represents a **numeric character** (`0–9`). Two `%` symbols mean two varying numbers will be generated in those positions (e.g., `00` through `99`).
  - **Space and** `!`: Trailing static characters.
- `o security_wordlist.txt`: Directs Crunch to save (**output**) the resulting wordlist to a text file named `security_wordlist.txt`.

Once the wordlist is ready, we'll use Hydra to brute-force the SSH service.

```
hydra -l marco -P security_wordlist.txt 10.48.149.185 ssh -V
-t 4
```

head pass_marco.txt

